

Sprint 1: Implementation and Initial Prototype

CP317 - A - Group 6

Laurier Club Room Booking System

Team Members:

Product Owner: ***Sawaab Anas***

Scrum Master: ***Muqadas Nazif***

Development Team: ***Manahil Bashir, Asmah Yasin Mohamed, Efetobore Salubi, Joleene Ismael***

Sprint Goal

The main goal of Sprint 01 was to get a simple version of HawkSpace up and running — something students could actually open and start using. We focused on building the essentials: letting users sign in, putting together the first key screens, and laying the groundwork for connecting everything to Firestore later on.

Originally, we planned to build a full registration and account-management system. But as the sprint progressed, we realized that it made more sense to keep things simple and only let users log in with their @mylaurier.ca email. This approach is closer to what a real campus system would do, removes the headache of running our own identity system, and still lets us personalize the experience by pulling the username from their email. It also helped us move faster and avoid building features that we'd eventually replace anyway once Firestore authentication is fully integrated.

The stories chosen for this sprint were the ones that let us deliver the most meaningful progress early: logging in, looking at available rooms, and making a booking. These features form the backbone of everything that comes later — whether that's storing bookings in the database, detecting scheduling conflicts, or adding an admin dashboard so campus staff can review and approve requests. Getting these core paths in place now helps us validate future decisions before we build more complex systems.

By the end of the sprint, a user can open the site, pass the @mylaurier.ca check, and navigate through the early flows to browse and choose rooms. Behind the scenes, we've also set up the structure needed to move booking data into Firestore and support additional roles (like admins) during the next sprint.

Implementation Progress

During Sprint 01, we successfully implemented core UI components and authentication logic, along with preliminary page navigation and room-interaction features. This sprint focused on building the application skeleton and enabling users to log in and browse available rooms. While certain final behaviors will be refined in Sprint 02, we now have a functioning flow from login → browse rooms → select rooms → create bookings stored locally.

1) Front-End UI & Component Structure

The UI was first tested using base HTML and CSS layouts. This allowed us to quickly verify content positioning and button flows before implementing the React-based component system. Once validated, we migrated logic into reusable components within React + Vite.

2) Authentication Phase 1 – Email-Only Login

Originally, we planned a full account-creation flow. However, after reviewing user needs, and considering our time-boxed sprint, we simplified authentication while still preserving security.

- ✓ Users now only need a **@mylaurier.ca email**
- ✓ Username is auto-extracted for display
- ✓ State is kept in context + local storage

3) Room System (Local Data Version)

Since full Firestore integration is planned for next sprint, we temporarily local-populated our room dataset. Rooms currently include ID, name, capacity, location, and availability.

Users can:

- ✓ Browse all rooms
- ✓ View room details
- ✓ Select a time duration and submit a booking

This booking is stored locally in browser storage.

The backend reservation flow will replace this in Sprint 02.

4) Full Firestore Preparation

Although full migration is not yet complete, Firestore is now:

- ✓ Initialized
- ✓ Connected to the app
- ✓ Permissions model discussion completed
- ✓ Initial room schema drafted

We discussed how Firestore rules will enforce security:

- Student users may book rooms
- Admin users can approve/deny/reschedule
- Prevent double-booking in same time window

Testing and Review

To verify Sprint 01 deliverables, the team conducted structured manual testing and iterative review throughout development. Because the project is still in an early phase, testing focused primarily on validating UI behaviors, functional flow, and data persistence rather than performance or stress testing. Development was carried out in a React + Vite environment, with Chrome DevTools used extensively to inspect state and ensure event handling occurred as expected.

Initial interface drafts were tested using static HTML/CSS before migrating to component-based React views. This allowed us to confirm layout assumptions and screen transitions early, reducing integration issues later. The switch to component-driven UI also improved testability by isolating concerns (e.g., login form, room list, booking modal).

Since Firestore integration is part of a later sprint, local storage was used as a temporary persistence mechanism. This allowed us to validate booking behavior realistically without relying on external dependencies. In addition, we implemented basic validation logic and tested edge cases, such as rejecting invalid emails and ensuring the system routed users appropriately based on authentication state.

Test Scenarios Performed

Scenario	Expected Outcome	Result
Load homepage	Home screen displays successfully	Pass
Enter valid @mylaurier.ca email	User is granted access and username extracted	Pass
Enter non-Laurier email	Error message displayed; access denied	Pass
Navigate to room list	Rooms render from local dataset	Pass
View room details	Displays room information	Pass
Book a room	Booking is successfully created	Pass
Booking persists on refresh	Local storage retains reservation	Pass
Attempt to book without login	Redirects to login screen	Pass

Acceptance criteria for Sprint 01 focused on ensuring that:

- Users can access the application using an @mylaurier.ca email,
- Navigation between pages works reliably,
- Rooms are presented in a consistent, readable format,
- A booking can be selected and retained locally,
- User identity persists across page refreshes.

Acceptance criteria for Sprint 01 focused on verifying that users could log in with a valid @mylaurier.ca email, navigate between screens, browse rooms in a consistent format, create bookings, and retain their session across refreshes. All items were demonstrated and accepted.

Visual styling was intentionally deprioritized this sprint. The team's focus was on ensuring clean, reliable user flow and functional data handling before committing to design. UI improvement and branding will be completed once core workflows are stable.

Firestore integration is planned for Sprint 02, where room and booking data will migrate from local storage to the cloud. Additionally, an administrative workflow—where staff can view, approve, and manage bookings—will be implemented in later sprints. This influenced several technical decisions, including how data structures were designed and how bookings are stored.

Key assumptions—such as temporary local storage, simplified time selection, and static room lists—were documented to guide future development. Overall, Sprint 01 delivered a stable functional prototype and establishes a strong foundation for database persistence, admin tooling, and future UI/UX enhancements.